CA-soln - Solution of Computer Architecture

Bachelors in Science Computer Science (Pokhara University)

1. Define Virtual Memory. Explain address mapping using pages in Virtual Memory.

Ans: Virtual memory is a feature of an operating system that enables a computer to be able to compensate shortages of physical memory by transferring pages of data from random access memory to disk storage.

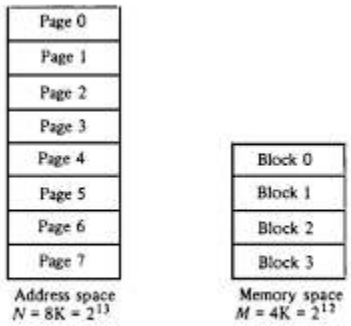
This process is done temporarily and is designed to work as a combination of RAM and space on the hard disk.

This means that when RAM runs low, virtual memory can move data from it to a space called a paging file. This process allows for RAM to be freed up so that a computer can complete the task.

Address Mapping Using Pages

A physical memory is broken down into groups of equal size called blocks or page frame and the groups of address space of the same size as block is called pages.

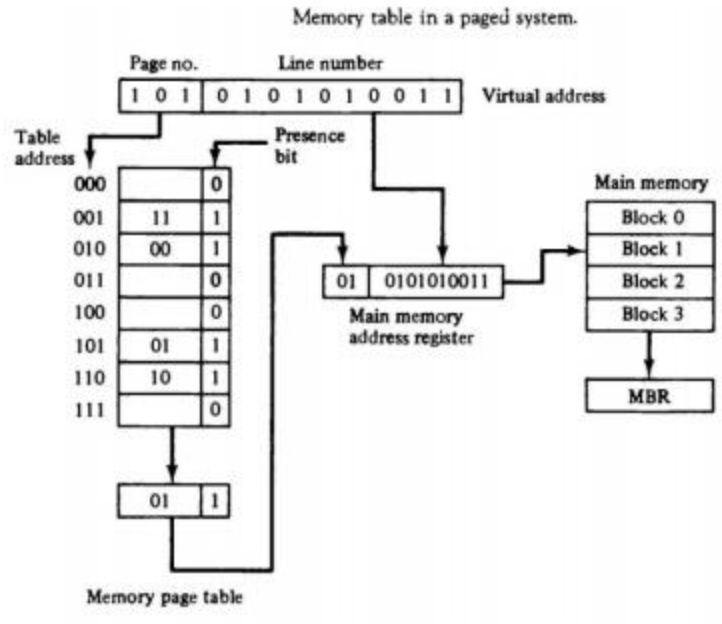
Consider a computer a with address space =8K and memory space=4K



Address space and memory space split into groups of 1K words.

Virtual address has 13 bits. Since each page consists of 2^{10}=1024 words, high-order 3 bits will specify one of 8 pages and low-order 10 bits gives the line address within the pages.

In computer 2^p words per page, p bits are used to specify a line address and remaining high-order bits of the virtual address specify the page number and an address for the memory page table. Line address in memory space and address space is same; only mapping required is from page number to a block number.



The memory page table consists of 8 words, one for each page. The address in the page table denotes page number and the content of word gives the block number where the page is stored in main memory.

Presence bit when 0 indicates page is not available in main memory and when 1 indicates that page has been transferred to main memory. Table shows that pages 1, 2, 5 & 6 are now available in main memory in blocks 3, 0, 1 & 2 respectively.

2. Explain with example how effective address is calculated.

Ans: Sir le deko pdf page 59

3. What is pipelining? Explain the different pipelining conflict and their solution.

Ans: Pipelining is a technique where multiple instructions are overlapped during execution. Pipeline is divided into stages and these stages are connected with one another to form a pipe like structure. Instructions enter from one end and exit from another end.

Pipeline Conflicts: There are some factors that cause the pipeline to deviate its normal performance. Some of these factors are given below:

1. **Timing Variations**: All stages cannot take same amount of time. This problem generally occurs in instruction processing where different instructions have different operand requirements and thus different processing time.

2. **Data Hazards**: When several instructions are in partial execution, and if they reference same data then the problem arises. We must ensure that next instruction does not attempt to access data before the current instruction, because this will lead to incorrect results.

3. **Branching**: In order to fetch and execute the next instruction, we must know what that instruction is. If the present instruction is a conditional branch, and its result will lead us to the next instruction, then the next instruction may not be known until the current one is processed.

4. **Interrupts**: Interrupts set unwanted instruction into the instruction stream. Interrupts effect the execution of instruction.

5. **Data Dependency**: It arises when an instruction depends upon the result of a previous instruction but this result is not yet available.

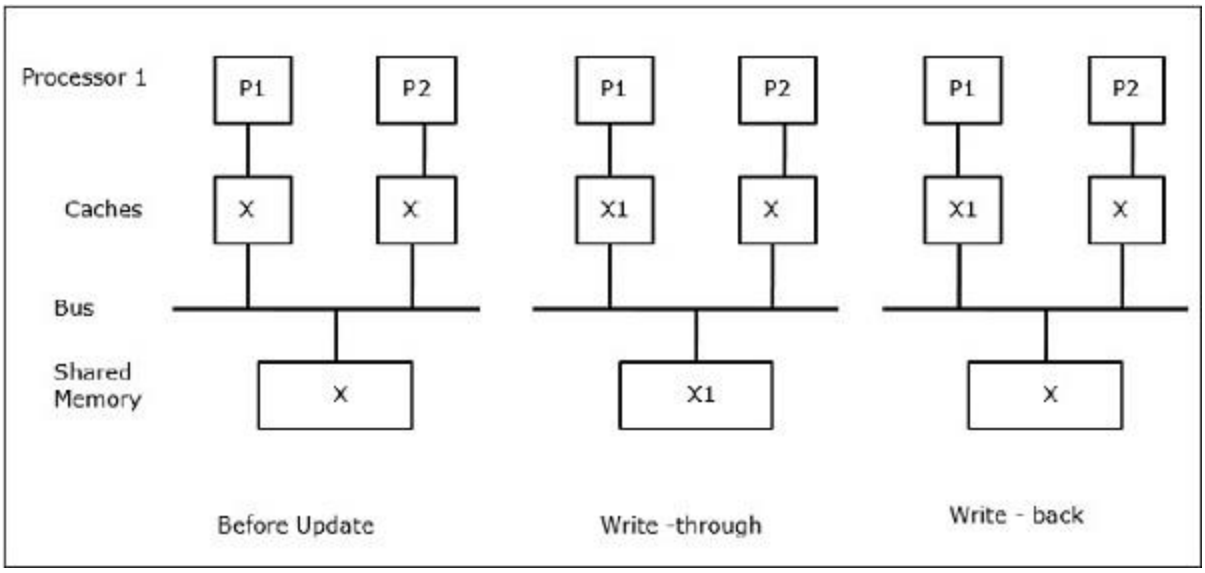
Solving conflicts:

1. **Hardware Interlocks** : The most straightforward method is to insert hardware interlocks. An interlock is a circuit that detects instructions whose source operands are destinations of instructions farther up in the pipeline. Detection of this situation causes the instruction whose source is not available to be delayed by enough clock cycles to resolve the conflict. This approach maintains the program sequence by using hardware to insert the required delays.
2. **Operand Forwarding**: Another technique called operand forwarding uses special hardware to detect a conflict and then avoid it by routing the data through special paths between pipeline segments. For example, instead of transferring an ALU result into a destination register, the hardware checks the destination operand, and if it is needed as a source in the next instruction, it passes the result directly into the ALU input, bypassing the register file. This method requires additional hardware paths through multiplexers as well as the circuit that detects the conflict.
3. **Prefetch Target Instruction**: One way of handling a conditional branch is to prefetch the target instruction in addition to the instruction following the branch. Both are saved until the branch is executed. If the branch condition is successful, the pipeline continues from the branch target instruction. An extension of this procedure is to continue fetching instructions from both places until the branch decision is made. At that time control chooses the instruction stream of the correct program flow.

4. What is cache coherence problem? Explain method of solving cache coherence.

Ans: In a multiprocessor system, data inconsistency may occur among adjacent levels or within the same level of the memory hierarchy. For example, the cache and the main memory may have inconsistent copies of the same object.

As multiple processors operate in parallel, and independently multiple caches may possess different copies of the same memory block, this creates **cache coherence problem**. **Cache coherence schemes** help to avoid this problem by maintaining a uniform state for each cached block of data.



Let X be an element of shared data which has been referenced by two processors, P1 and P2. In the beginning, three copies of X are consistent. If the processor P1 writes a new data X1 into the cache, by using **write-through policy**, the same copy will be written immediately into the shared memory. In this case, inconsistency occurs between cache memory and the main memory. When a **write-back policy** is used, the main memory will be updated when the modified data in the cache is replaced or invalidated.

Solution of cache coherence

Software based cache coherence solution

→ It is compiler based or with run time system support & it works with or without hardware assit. It has tough problem because perfect information needed in the presence of memory aliasing & explict parallelism. Thus need to focus on hardware based solutions as they are more common.

→ A simple scheme is to disallow private cache for each processor & have a shared cache memory associate with main memory. Every data access is made to the shared cache. This method voilates the principe of closeness of CPU to cache & increases the average memory access time. In this case this scheme solves the problem by avoiding it.

→ For performance consideration it is desired to attach private cache to each processor. One scheme that has been used allows only non-shared and read only data to be stored in caches. Such items are called cacheable. Shared writable data. Are non-cacheable. The compiler must tag data as either cacheable or non cacheable, and the system hardware makes sure only cacheable data are stored in cache. The non cacheable data remains in the main memory. This method restricts the type of data stored in the cache and introduces an extra software overhead that may degrade performance.

→ A scheme allows cachable data to exist in atleast one cache is a method that employs a centralized global in its compiler. The status of memory blocks is stored in the central global table. Each block is identified as read only only or Read & write. All caches can have copies of blocks identified as Ro. only one cache can have a copy of an Rw block. Thus if the data are updated in the cache with an Rw block, the other cache are not affected because they deen donot have copy of this block.

Hardware based cache coherence solutions
i) shared caches vs. snoopy schemes
ii) write through vs write back (ownership based) protocols
iii) Update vs invalidation protocols

Snoopy cache coherence schemes.
→ A distributed cache coherence scheme based on the notation of a snoop that watches all activity on a global bus, or is informed about such activity by some global broadcast mechanism. This is the most commonly used method in commercial multiprocess.

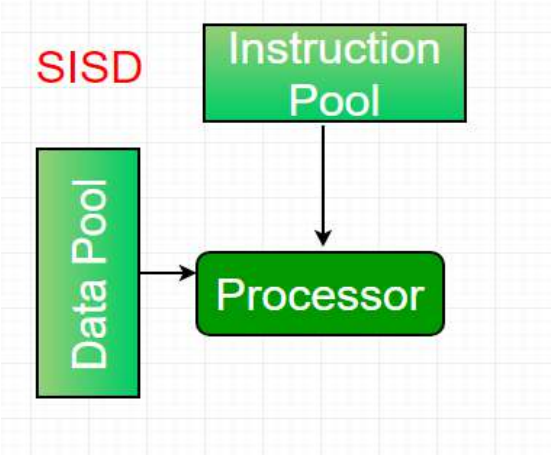
5. What are the various interconnection structure used for multi-processing system? Explain. [10]

Ans: https://www.geeksforgeeks.org/time-shared-bus-interconnection-structure-in-multiprocessor-system/

6. Explain Flynn's Classification of computer system.

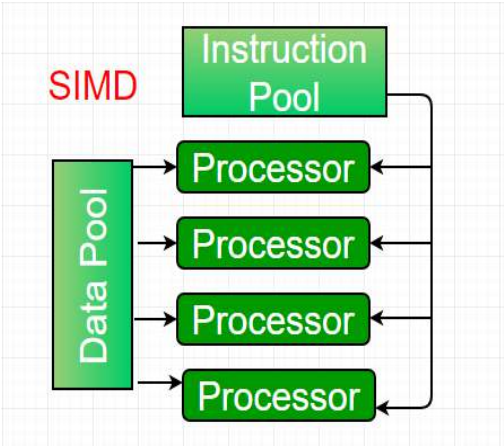
Ans: Flynn's classification –

1. **Single-instruction, single-data (SISD) systems –**
   An SISD computing system is a uniprocessor machine which is capable of executing a single instruction, operating on a single data stream. In SISD, machine instructions are processed in a sequential manner and computers adopting this model are popularly called sequential computers. Most conventional computers have SISD architecture. All the instructions and data to be processed have to be stored in primary memory.



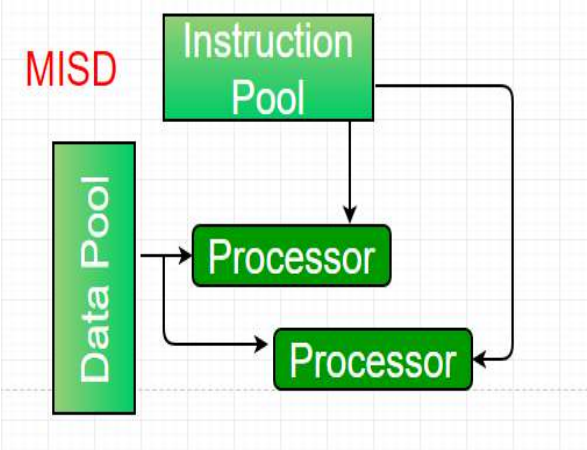
   The speed of the processing element in the SISD model is limited(dependent) by the rate at which the computer can transfer information internally. Dominant representative SISD systems are IBM PC, workstations.

2. **Single-instruction, multiple-data (SIMD) systems –**
   An SIMD system is a multiprocessor machine capable of executing the same instruction on all the CPUs but operating on different data streams. Machines based on an SIMD model are well suited to scientific computing since they involve lots of vector and matrix operations. So that the information can be passed to all the processing elements (PEs) organized data elements of vectors can be divided into multiple sets(N-sets for N PE systems) and each PE can process one data set.



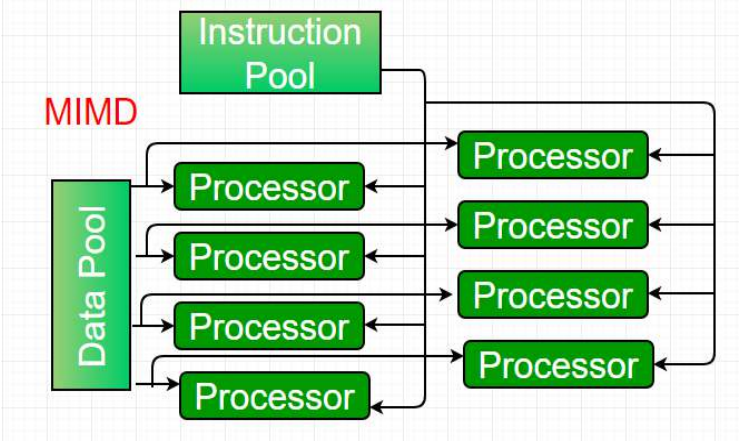
Dominant representative SIMD systems is Cray's vector processing machine.

3. **Multiple-instruction, single-data (MISD) systems –**
   An MISD computing system is a multiprocessor machine capable of executing different instructions on different PEs but all of them operating on the same dataset .



   Example Z = sin(x)+cos(x)+tan(x)
   The system performs different operations on the same data set. Machines built using the MISD model are not useful in most of the application, a few machines are built, but none of them are available commercially.

4. **Multiple-instruction, multiple-data (MIMD) systems –**
   An MIMD system is a multiprocessor machine which is capable of executing multiple instructions on multiple data sets. Each PE in the MIMD model has separate instruction and data streams; therefore machines built using this model are capable to any kind of application. Unlike SIMD and MISD machines, PEs in MIMD machines work asynchronously.



https://www.geeksforgeeks.org/computer-architecture-flynns-taxonomy/

7. **What is DMA? Explain DMA controller with different operating mode.**

Ans: **Direct Memory Access** (DMA) transfers the block of data between the *memory* and *peripheral devices* of the system, **without the participation** of the **processor**. The unit that controls the activity of accessing memory directly is called a **DMA controller**.

DMA controller has four modes for data transfer:

1. Single Byte Transfer Mode/ Cycle Stealing

- Once the DMAC becomes the bus master, it will transfer only ONE BYTE and return the bus back to the microprocessor. As soon as the microprocessor performs one bus cycle, DMAC will once again take the bus back from the microprocessor.
- Both DMAC and microprocessor are constantly stealing bus cycles from each other. It is the most popular method of DMA, because it keeps the microprocessor active in the background.
- After a byte is transferred, the CAR and CWCR are adjusted accordingly. The system bus is returned to the µP. For further bytes to be transferred, the DREQ line must go active again, and then the entire operation is repeated.

2. Block Transfer Mode.

- In this mode, the DMAC is programmed to transfer all the bytes in one complete DMA operation. After a byte is transferred, the CAR and CWCR are adjusted accordingly.
- The system bus is returned to the µP, only after all the bytes are transferred. i.e. TC is reached or EOP signal is issued. It is the fastest form of DMA but keeps the microprocessor inactive for a long time.
- The DREQ signal needs to be active only in the beginning for requesting the DMA service initially. Thereafter DREQ can become low during the transfer.

3. Demand Transfer Mode

- It is very similar to Block Transfer, except that the DREQ must active throughout the DMA operation.

- If during the operation DREQ goes low, the DMA operation is stopped and the busses are returned to the μP.
- In the meantime, the μP can continue with its own operations. Once DREQ goes high again, the DMA operation continues from where it had stopped.

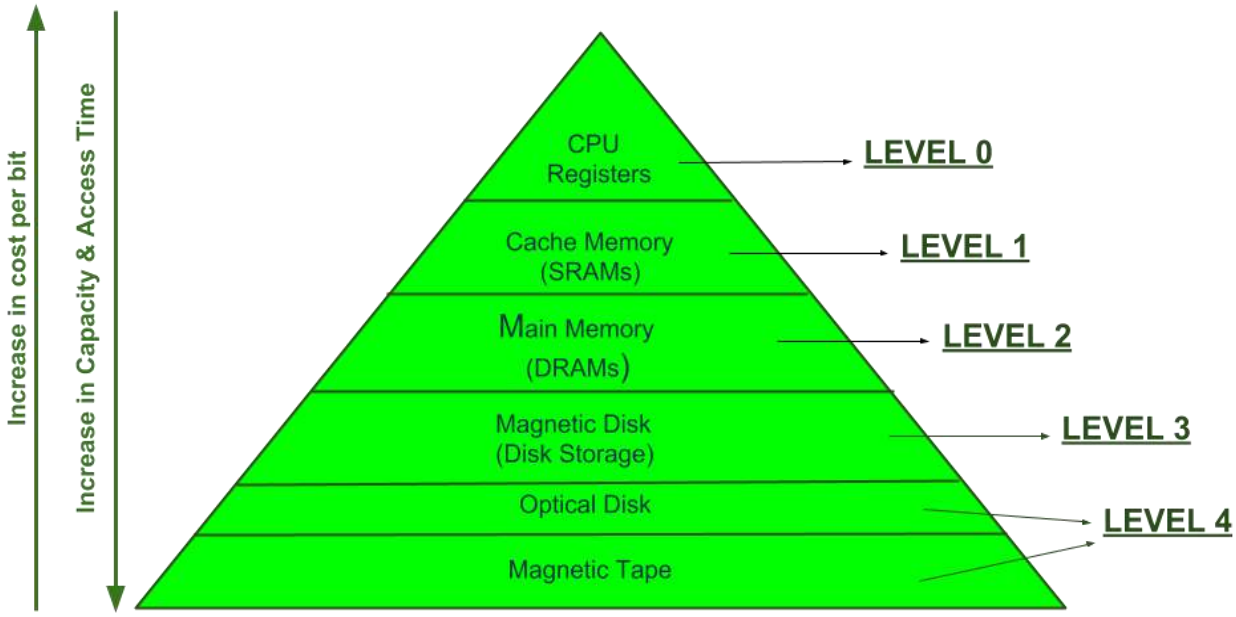
4. Cascade Transfer Mode

- In this mode, more than one DMACs are cascaded together. It is used to increase the number of devices interfaced to the μP. Here we have one Master DMAC, to which one or more Slave DMACs are connected.
- The Slave gives HRQ to the Master on the DREQ of the Master, and the Master gives HRQ to the μP on the HOLD of the μP.

Short Note:

RISC	**CISC**
Focus on software	Focus on hardware
Uses only Hardwired control unit	Uses both hardwired and microprogrammed control unit
Transistors are used for more registers	Transistors are used for storing complex Instructions
Fixed sized instructions	Variable sized instructions
Can perform only Register to Register Arithmetic operations	Can perform REG to REG or REG to MEM or MEM to MEM
Requires more number of registers	Requires less number of registers
Code size is large	Code size is small
An instruction executed in a single clock cycle	Instruction takes more than one clock cycle
An instruction fit in one word	Instructions are larger than the size of one word

Memory Hierarchy: In the Computer System Design, Memory Hierarchy is an enhancement to organize the memory such that it can minimize the access time. The Memory Hierarchy was developed based on a program behavior known as locality of references.The figure below clearly demonstrates the different levels of memory hierarchy :



MEMORY HIERARCHY DESIGN

This Memory Hierarchy Design is divided into 2 main types:

1. **External Memory or Secondary Memory –**
   Comprising of Magnetic Disk, Optical Disk, Magnetic Tape i.e. peripheral storage devices which are accessible by the processor via I/O Module.
2. **Internal Memory or Primary Memory –**
   Comprising of Main Memory, Cache Memory & CPU registers. This is directly accessible by the processor.

We can infer the following characteristics of Memory Hierarchy Design from above figure:

1. **Capacity:**
   It is the global volume of information the memory can store. As we move from top to bottom in the Hierarchy, the capacity increases.
2. **Access Time:**
   It is the time interval between the read/write request and the availability of the data. As we move from top to bottom in the Hierarchy, the access time increases.
3. **Performance:**
   Earlier when the computer system was designed without Memory Hierarchy design, the speed gap increases between the CPU registers and Main Memory due to large difference in access time. This results in lower performance of the system and thus, enhancement was required. This enhancement was made in the form of Memory

Hierarchy Design because of which the performance of the system increases. One of the most significant ways to increase system performance is minimizing how far down the memory hierarchy one has to go to manipulate data.

4. **Cost per bit:**
   As we move from bottom to top in the Hierarchy, the cost per bit increases i.e. Internal Memory is costlier than External Memory.

Logical Microoperation:

Logic operations are binary micro-operations implemented on the bits saved in the registers. These operations treated each bit independently and create them as binary variables.

For example, the exclusive-OR micro-operation with the contents of two registers R1 and R2 is denoted by the statement

P: R1←R1⊕⊕R2

It determines a logic micro-operation to be implemented on the single bits of the registers supported that the control variable P = 1. Consider that each register has four bits. Let the content of R1 be 1010 and the content of R2 be 1100.

The exclusive-OR micro-operation stated above represent the following logic computation −

1010 Content of R1

1100 Content of R2

0110 Content of R1 after P = 1

The content of R1, after the implementation of the micro-operation, is similar to the bit-by-bit exclusive-OR operation on pairs of bits in R2 and previous values of R1.

Special Symbols

Special symbols will be approved for the logic micro-operations OR, AND, and complement, to categorize them from the matching symbols that can define Boolean functions. The symbol V can indicate an OR micro-operation and the symbol can indicate an AND micro-operation.

The complement micro-operation is similar to the 1's complement and supports a bar on the highest of the symbol that indicates the registered name. There are various symbols, and it will be applicable to differentiate between a logic micro-operation and a control (or Boolean) function.

There is another sense for supporting two sets of symbols can that recognize the symbol +, when can symbolize arithmetic plus, from a logic OR operation. Although the + symbol has two meanings, it will be available to determine between them by observing where the symbol appears.

When the symbol + appears in a micro-operation, it will indicate an arithmetic plus. When it appears in a control (or Boolean) function, it will isndicate an OR operation. We cannot use it to symbolize an OR micro-operation.

For example, in the statement

P+Q: R1←R2+R3,R 4←R5V R6

The + between P and Q is an OR operation between two binary variables of a control function. The + between R2 and R3 determines an add micro-operation. The OR micro-operation is named by the symbol V between registers R5 and R6.